# MouseCatcher — Technical Documentation (v1.9 baseline)

Author: Bo Gärdmark (SM6FIE), Gothenburg, Sweden

Created: 2026-02-28 19:59:37 CET

License: MIT (see LICENSE in repository)

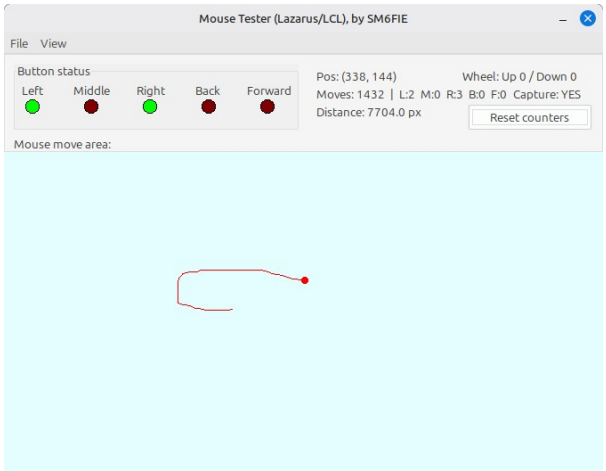
## 1. Purpose and scope

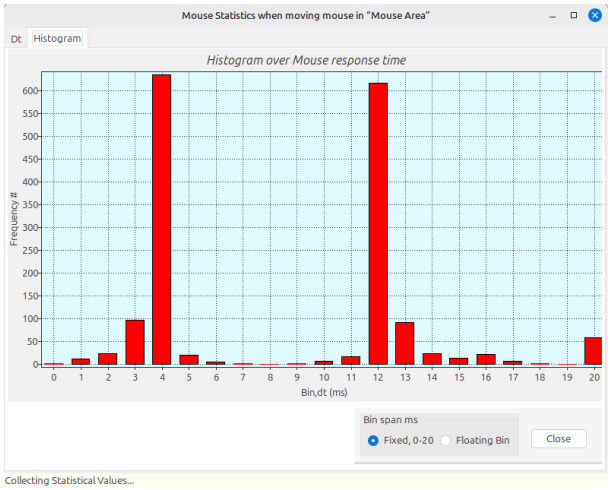
MouseCatcher is a Lazarus/LCL application for Linux that measures mouse behavior in real time: cursor movement timing (dt), click activity (including XButton1/XButton2), and statistical stability (jitter, outliers) with plots (time series + histogram).

This document describes the architecture and implementation decisions for the v1.9 code line (working baseline). It is intended for developers who want to understand, maintain, or extend the program.

## 2. Key features

- Real-time capture of mouse movement inside a defined capture area (PaintBox).
- Button LEDs and counters for Left / Middle / Right / Back (XButton1) / Forward (XButton2).
- Wheel up/down counters.
- dt timing statistics: mean, median, P95, max; drop counters (e.g. >25 ms, >50 ms).
- Estimated polling rate (Hz) based on dt distribution (robust estimator).
- Plots: dt over sample index (ChartDt/serDt) and histogram distribution (ChartHist/serHist).
- Histogram bin span modes via rgBinSpan (Fixed 0–20 ms / Floating span).
- Separation of main UI (capture + live counters) and statistics UI (charts + metrics).
- Code style: Hungarian-ish prefixes (i/d/c/s/b), comment blocks per project standard.




## 3. Environment and dependencies

Validated setup (example):

- Linux Mint 22.2 (Zara), base Ubuntu 24.04 (noble).
- Lazarus 4.4, Free Pascal 3.2.x, LCL (GTK2/Qt depending on your build).

- TAChart package (for TChart, TLineSeries, histogram series).
- Standard LCL controls: TForm, TPaintBox, TTimer, TPanel/TGroupBox, TMenuItem, etc.

*Important: if you see runtime errors like “Class 'TChart' not found”, it usually means TAChart is not included/registered in the project build or the .lfm/.lrs resources were compiled without the package. In Lazarus, ensure TAChart is installed and the project has the relevant chart units in its uses list.*

## 4. Repository layout

Typical files/folders in the repository:

- MouseCatcher.lpi / .lpr — Lazarus project files
- umousecatcher.pas / .lfm — main form (capture UI)
- ustats.pas / .lfm — statistics form (charts + metrics)
- uinputstats.pas — shared statistics collection/processing helpers (dt ring buffer, etc.)
- docs/USER_GUIDE.md — end-user instructions (rendered via RichMemo in-app)
- docs/DEV_NOTES.md — developer notes (build, packages, design decisions)
- README.md — project landing page for GitHub

## 5. Data flow overview

At runtime the program follows this simplified data flow:

- Main form receives mouse move events (PaintBoxMouseMove).
- Compute dt = time_now − time_prev for movement samples (only while inside capture area).
- Push dt into a ring buffer (Cardinal ms) in uInputStats.
- Stats form timer (TmrUpdate) periodically pulls a snapshot from the ring buffer.
- Stats form recomputes summary statistics and refreshes charts.
- Reset action clears counters + ring buffer + message/status flags.

## 6. Timing measurement (dt)

dt is the time between consecutive mouse movement samples while the cursor is inside the capture area. Mouse movement sampling is ultimately limited by the OS event delivery and USB polling. On Linux, the event loop and scheduling jitter can produce occasional large dt spikes; these are treated as outliers and summarized via percentiles and drop counters.

Implementation notes:

- Use a monotonic clock source. In Lazarus/FPC, GetTickCount64 is commonly used; it is monotonic on modern Linux.
- Store dt in milliseconds as Cardinal (0..4,294,967,295).
- Ignore dt=0 samples if desired (depends on event coalescing). v1.9 keeps them to show OS coalescing behavior.
- Use a fixed maximum ring buffer size to bound memory and ensure stable performance.

## 7. Ring buffer design (uInputStats)

The dt samples are stored in a preallocated ring buffer (circular array). This avoids dynamic growth and keeps Up-date-Summary-And-Plot fast and predictable.

Recommended ring buffer behavior:

- Constants: DT_RING_MAX defines capacity (e.g. 4096 or 8192).
- Write index increments with wrap-around.
- Count saturates at DT_RING_MAX once full.
- Snapshot function copies samples out in chronological order (oldest → newest).

Typical snapshot pseudo-code:

```
function GetDtSnapshot(var DtMsArr: array of Cardinal): Integer;
var
iIdx, iCount, iStartIdx: Integer;
begin
// iCount := number of valid samples (<= DT_RING_MAX)
// iStartIdx := oldest sample index
// copy iCount samples in order into DtMsArr[0..iCount-1]
end;
```

## 8. Statistics computation (ustats.pas)

The Stats form periodically computes summary metrics from a snapshot. Computation is designed to be robust against outliers and fast enough for frequent refresh (e.g. 5–20 Hz).

Metrics in v1.9:

- Sample count (N).
- Mean dt (ms) as Double.
- Median dt (ms) as Cardinal (50th percentile).
- P95 dt (ms) as Cardinal (95th percentile).
- Max dt (ms).
- Drop counts: number of samples exceeding thresholds (e.g. >25 ms and >50 ms).
- Estimated polling rate (Hz) from median dt (or robust alternative).

*Sorting: Lazarus/FPC does not provide Delphi's TArrayHelper. Use a local quicksort/mergesort on a Cardinal array or use FPC's generic sort helpers if available. For simplicity and portability, many projects implement a small in-unit QuickSortCardinal.*

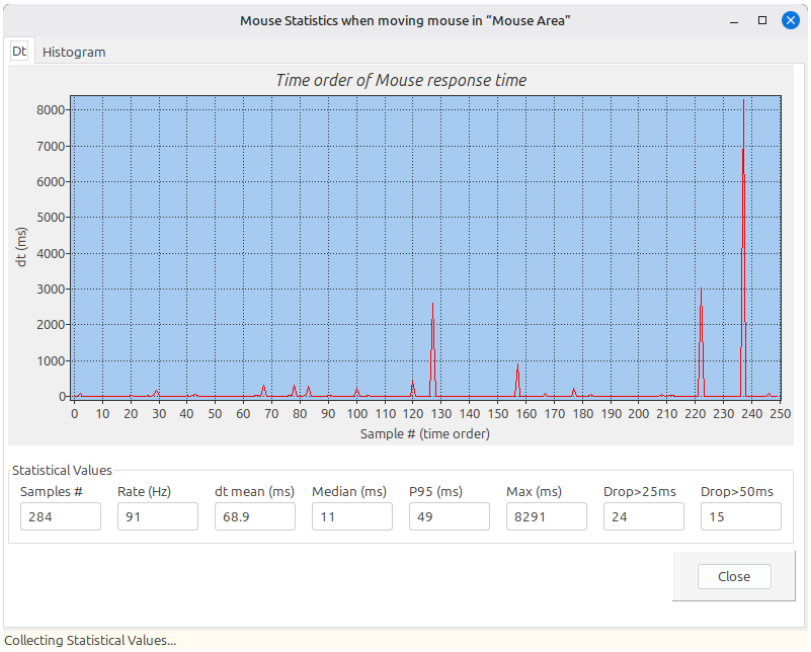
## 9. Plotting

Two charts are used on the Stats form:

- ChartDt (dt over sample index) with serDt (TLineSeries). X-axis legend text: "Sample #".
- ChartHist (histogram) with serHist. Bin span controlled by rgBinSpan.

Histogram bin modes:

- Fixed mode: 0–20 ms with fine bins (useful for normal operation).
- Floating mode: span adapts to current max dt (useful when spikes occur).
- Bin count should be stable (e.g. 100–250 bins) to keep visuals comparable.



## 10. UI behavior and messaging

A status message is shown to guide the user:
1) “No motion timing samples yet. Move cursor in the main plot.”
2) “Collecting Statistical Values…” once at least one dt sample exists.

In v1.9, a boolean flag is used so the status text is only updated when needed (avoids repeated string assignments during fast timer updates).

## 11. Reset behavior

Reset is triggered from the main form (Reset counters). It should:

- Clear button and wheel counters.
- Clear distance accumulator and movement counters.
- Clear dt ring buffer and timing state (previous timestamp).
- Reset status message state (so Stats shows message 1 again).
- Optionally clear charts on Stats form (or let next timer tick redraw empty).

## 12. Performance considerations

- Avoid per-event allocations inside mouse-move handlers.
- Compute dt only while inside capture area; ignore moves outside.
- Keep Stats timer interval moderate (e.g. 200–500 ms) for heavy charts; allow user to change if needed.
- Rebuild histogram from a snapshot; do not update per sample.
- When using sorting, prefer sorting a copied array; keep ring buffer untouched.

## 13. Known limitations

- dt reflects OS event delivery, not raw USB polling directly (coalescing can occur).
- Large spikes may be caused by background load, power management, or compositor delays.
- Comparing mice requires identical USB port, hub, and power settings for fair results.
- Wayland vs X11 can affect event timing; document your desktop session when reporting results.

## 14. Suggested future extensions

- Polling rate estimator improvements (use mode/peak of histogram rather than median).
- Jitter metrics: standard deviation, MAD (median absolute deviation), interquartile range.
- Per-button click latency measurement (down→up durations) and click-bounce detection.
- Export: CSV of dt samples and summary report for sharing.
- Profiles: save/load measurement sessions.
- Optional hardware info: USB device details (lsusb) captured into the report.

***Document Version Control***
*Version:* *2*
*Author:* *BG*
*Length:* *7876*
*Path:* */home/bag/projects/MouseCatcher/docs/*